

Bayesian Knowledge Tracing (BKT) Implementation Report

Overview

In this work, a Bayesian Knowledge Tracing (BKT) pipeline was implemented in Python using the pyBKT library.

The goal was to model student knowledge acquisition and predict whether future student answers would be correct or incorrect based on previous interactions.

The implementation was tested on several educational datasets, including the Physics dataset.

Data Preprocessing

Before training the model, multiple preprocessing steps were applied to the datasets.

Selected Columns

Only the columns required for BKT training were used:

- Student ID
- Skill / Knowledge Component
- Student Answer
- Timestamp

Missing Values Removal

Rows containing missing values (NaN) were removed from the dataset.

Filtering Valid Answers

Only rows containing:

- correct
- incorrect

were kept.

Other answer types such as hints or unknown responses were removed because BKT requires binary correctness labels.

Removing Multiple Skills

Rows containing multiple skills in the same interaction (e.g. skillA~~skillB) were removed.

Reason:

The standard BKT model assumes exactly one skill per interaction.

Chronological Sorting

The dataset was sorted using:

- student ID
- timestamp

This step is important because BKT is a sequential model and must observe student interactions in temporal order.

Label Conversion

Student answers were converted into numerical values:

- correct → 1
- incorrect → 0

Data Splitting

Instead of using a standard random split, the dataset was divided using `GroupShuffleSplit`.

Reason:

The same student should not appear in both the training and testing datasets.

This prevents data leakage and produces a more realistic evaluation.

Split configuration:

- Training set: 80%
- Test set: 20%

BKT Model Training

The BKT model was trained using the pyBKT library.

The model learns several hidden parameters for each skill:

- Prior knowledge
- Learning probability
- Guess probability
- Slip probability

The model was initialized using a fixed random seed to ensure reproducibility.

Prediction Phase

After training, predictions were generated on the test dataset.

The model produced two important outputs:

Correct Predictions

Probability that the next student answer will be correct.

Example:

0.91 → high probability of a correct answer

0.15 → low probability of a correct answer

State Predictions

Probability that the student has mastered the skill.

This value represents the estimated hidden knowledge state of the learner.

ROC Curve Analysis

A ROC Curve was generated to evaluate the classification performance of the model.

The ROC curve compares:

- True Positive Rate (Sensitivity)
- False Positive Rate

The Area Under the Curve (ROC AUC) was used as a global evaluation metric.

A higher ROC AUC indicates better discrimination between correct and incorrect answers.

Optimal Threshold Selection

Instead of using a fixed threshold of 0.5, an optimal threshold was computed from the ROC curve.

The optimal threshold was selected using:

$\text{argmax}(\text{TPR} - \text{FPR})$

This improves classification quality, especially for imbalanced datasets.

Precision-Recall Curve

A Precision-Recall (PR) curve was also generated.

This evaluation is especially useful for educational datasets where the number of correct and incorrect answers is imbalanced.

The PR curve visualizes the trade-off between:

- Precision
- Recall

Evaluation Metrics

Several evaluation metrics were computed:

Accuracy

Measures the percentage of correct predictions.

ROC AUC

Measures the model's ability to distinguish between correct and incorrect answers independently of the classification threshold.

Confusion Matrix

The confusion matrix summarizes:

- True Positives
- True Negatives
- False Positives
- False Negatives

It provides detailed insight into model errors.

Classification Report

The classification report includes:

- Precision
- Recall
- F1-score

for both classes.

Results

The Physics dataset achieved strong performance:

- Accuracy $\approx 85\%$
- ROC AUC ≈ 0.89

These results indicate that the BKT model successfully captured student learning behavior and produced reliable predictions.

Conclusion

A complete BKT pipeline was successfully implemented, including:

- preprocessing
- chronological ordering
- grouped train/test splitting
- model training
- prediction
- ROC analysis
- Precision-Recall analysis
- visualization

- evaluation metrics

The final implementation provides a robust framework for analyzing student learning and predicting future performance using Bayesian Knowledge Tracing.